

ML307C

SSL/TLS开发指导手册

版本：V1.0.0

发布日期：2025/3/6

服务与支持

如果您有任何关于模组产品及产品手册的评论、疑问、想法，或者任何无法从本手册中找到答案的疑问，请通过以下方式联系我们。

OneMO官网： onemo10086.com

邮箱： SmartModule@cmiot.chinamobile.com

客户服务热线： 400-110-0866



中国移动
China Mobile

文档声明

注意

本手册描述的产品及其附件特性和功能，取决于当地网络设计或网络性能，同时也取决于用户预先安装的各种软件。由于当地网络运营商、ISP，或当地网络设置等原因，可能也会造成本手册中描述的全部或部分产品及其附件特性和功能未包含在您的购买或使用范围之内。

责任限制

除非合同另有约定，中移物联网有限公司对本文档内容不做任何明示或暗示的声明或保证，并且不对特定目的适销性及适用性或者任何间接的、特殊的或连带的损失承担任何责任。

在适用法律允许的范围内，在任何情况下，中移物联网有限公司均不对用户因使用本手册内容和本手册中描述的产品而引起的任何特殊的、间接的、附带的或后果性的损坏、利润损失、数据丢失、声誉和预期的节省而负责。

因使用本手册中所述的产品而引起的中移物联网有限公司对用户的最大赔偿（除在涉及人身伤害的情况中根据适用法律规定的损害赔偿外），不应超过用户为购买此产品而支付的金额。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。公司保留随时修改本手册中任何信息的权利，无需进行提前通知且不承担任何责任。

商标声明



为中国移动注册商标。

本手册和本手册描述的产品中出现的其他商标、产品名称、服务名称和公司名称，均为其各自所有者的财产。

进出口法规

出口、转口或进口本手册中描述的产品（包括但不限于产品软件和技术数据），用户应遵守相关进出口法律和法规。

隐私保护

关于我们如何保护用户的个人信息等隐私情况，请查看相关隐私政策。

操作系统更新声明

操作系统仅支持官方升级；如用户自己刷非官方系统，导致安全风险和损失由用户负责。

固件包完整性风险声明

固件仅支持官方升级；如用户自己刷非官方固件，导致安全风险和损失由用户负责。

版权所有©中移物联网有限公司。保留一切权利。

本手册中描述的产品，可能包含中移物联网有限公司及其存在的许可人享有版权的软件，除非获得相关权利人的许可，否则，非经本公司书面同意，任何单位和个人不得擅自摘抄、复制本手册内容的部分或全部，并以任何形式传播。



中国移动
China Mobile

关于文档

修订记录

版本	描述
V1.0.0	初版



中国移动
China Mobile

目录

服务与支持.....	ii
文档声明.....	iii
关于文档.....	v
1. 概述.....	7
1.1. 适用范围.....	8
1.2. 写作目的.....	9
2. 功能说明.....	10
3. API说明.....	11
4. 应用指导.....	12
4.1. 操作流程示例.....	13
5. 编程设计注意.....	20
6. 附录.....	21



中国移动
China Mobile

1. 概述

本文档介绍了OpenCPU SDK的SSL/TLS功能，包括API说明、应用指导和编程设计注意等内容，以便于用户参照文档和SDK中的示例程序完成应用程序编写。



中国移动
China Mobile

1.1. 适用范围

Table 1. 适用模组

模组型号	模组子型号
ML307C	ML307C-DC-CN

1.2. 写作目的

本文档介绍了SSL接口函数使用和注意事项，旨在为开发者使用SDK进行应用程序开发时提供参考。由于SSL接口用于和其他模块配合完成加密传输，所以编写此文档说明如何使用SSL接口。

2. 功能说明

OpenCPU SDK提供的SSL功能包括连接、关闭、读取和发送，提供的接口可供其他模块做加密传输使用（例如MQTT或HTTPS），也可配合socket接口做TCP加密传输使用。在资源空间足够的情况下，最多可同时配置6路通道。



中国移动
China Mobile

3. API说明

详细的函数说明可参考头文件cm_ssl.h中注释说明。

相关文件路径：

- API头文件路径为include/cm_iot/cm_ssl.h
- 示例文件路径为examples/ssl/src/cm_demo_ssl.c

接口功能说明和建议使用步骤：

1. 使用cm_ssl_setopt()接口进行对应通道的参数配置，配置操作符可见cm_ssl_param_type_e枚举信息，默认无身份验证，默认握手超时时间300秒。
2. 使用cm_ssl_conn()接口进行握手连接，该接口需要在socket连接成功后再执行，并将socket描述符作为入参，该接口为阻塞接口，根据返回值即可判断连接结果，握手超时时长由cm_ssl_setopt()配置，具体使用方法请参考使用示例。
3. 使用cm_ssl_write()接口进行数据发送。
4. 使用cm_ssl_read()接口进行数据接收，返回的读取长度是解密后的长度，与socket接口查询到的数据长度不同，读取超时时长由cm_ssl_conn()执行时的入参决定，具体使用方法请参考使用示例。
5. 使用cm_ssl_check_pending()接口查询缓存区是否还有未读数据。
6. 使用cm_ssl_get_bytes_avail()接口查询缓存区可读数据长度。
7. 使用cm_ssl_close()接口关闭SSL连接，关闭连接时，需要先关闭SSL连接才能关闭socket连接。

4. 应用指导

提供SSL接口配合socket接口实现TCP加密传输的示例程序以及配合其他模块实现加密传输的说明。

4.1. 操作流程示例

本节主要提供使用SSL接口实现加密传输的示例。主要介绍使用SSL接口和ASOCKET接口实现TCP加密传输，首先需要先用ASOCKET接口连接上平台，然后进行SSL握手连接。握手过程中ASOCKET回调函数会触发接收事件，此时不能进行数据读取。连接成功后根据ASOCKET回调函数触发的事件进行业务，触发接收事件时使用cm_ssl_read()进行读取，同时可以使用cm_ssl_write()进行数据发送。

SSL配置

使用cm_ssl_setopt()接口，选择指定通道完成验证模式、握手超时时间、证书等数据的配置。

```
/* 以www.baidu.com根证书为例 */
static char *http_ca = "-----BEGIN CERTIFICATE-----\r\n\"
"MIIDdTCCA12gAwIBAgILBAAAAAABFUtaw5QwDQYJKoZIhvcNAQEFBQAwVzELMAkG\r\n\"
"A1UEBhMCQkUxGTAXBgNVBAoTEEdsb2JhbFNpZ24gbnYtc2ExEDAOBgNVBAsTB1Jv\r\n\"
"b3QgQ0ExGzAZBgNVBAMTEkdsb2JhbFNpZ24gUm9vdCBDQTAeFw05ODA5MDExMjAw\r\n\"
"MDBaFw0yODAxMjg0MjAwMDBaMFcxZCZAJBgNVBAYTAkFMRkwFwYDVQQKEXBHbG9i\r\n\"
"YWxTaWduIG52LXNhMRAwDgYDVQQLEwdSb290IENBMRSwGQYDVQQDEXJHbG9iYWxT\r\n\"
"aWduIFJvb3Q0EwggEiMA0GCSqGSIb3DQEBAAQAA4IBDwAwggEKAoIBAQDaDuaZ\r\n\"
"jc6j40+Kfvxi4Mla+piH/EqsLmVEQS98GPR4mdmxzdzxtIK+6NiY6arymAZavp\r\n\"
"xy0Sy6scTHAHOt0KMM0VjU/43dSMUBUc71DuxC73/OIS8pF94G3VNTCOXkNz8kHp\r\n\"
"1Wrjsok6Vjk4bwY8iGlbKk3Fp1S4blnMm/k8yuX9ifUSPJJ4ltbcdG6TRGHRjcdG\r\n\"
"snUOhugZitVtbNV4FpWi6cgKOOvyJBNPc1STE4U6G7weNLWLBYY5d4ux2x8kasJ\r\n\"
"U26Qzns3dLlwr5EiUWMWwa6xrEmCMgZK9FGqjWZCrXgzT/LCrBbBIDSgeF59N8\r\n\"
"9iFo7+ryUp9/k5DPAGMBAAGjQjBAMA4GA1UdDwEB/wQEAwIBBjAPBgNVHRMBAf8E\r\n\"
"BTADAQH/MB0GA1UdDgQWBRRge2YaRQ2XyolQL30EzTSO/z9SzANBgkqhkiG9w0B\r\n\"
"AQUFAAOCAQEAnPnfE920I2/7LqivjTFKDK1fPxsncwrvQmeU79rXqoRSLbLCKOz\r\n\"
"yj1hTdNGCBm+w6DjY1Ub8rrvrTnhQ7k4o+Yvii776BQVvnGCv04zcQLcFGUL5gE\r\n\"
"38NfINUvYRRBnMRddWQVDF9VM0yGj/8N7yy5Y0b2qvzfVgn9LhJlZJrglfCm7ymP\r\n\"
"AbEVtQwdpf5pLGkkeB6zpxxxYu7KyJesF12KwvhHhm4qxFYxldBniYur+WymXUad\r\n\"
"DKqC5JIR3XC321Y9YeRq4VzW9v493kHMB65jUr9TU/Qr6cf9tveCX4XSQRjbgBME\r\n\"
"HMUfpIBvFSDJ3gyIch3WZlXi/EjKJSzP4A==\r\n\"
"-----END CERTIFICATE-----\r\n\"
"-----BEGIN CERTIFICATE-----\r\n\"
"MIIEATCCA1GgAwIBAgILBAAAAAABRE7wQkcdQYJKoZIhvcNAQELBQAwVzELMAkG\r\n\"
"A1UEBhMCQkUxGTAXBgNVBAoTEEdsb2JhbFNpZ24gbnYtc2ExEDAOBgNVBAsTB1Jv\r\n\"
"b3QgQ0ExGzAZBgNVBAMTEkdsb2JhbFNpZ24gUm9vdCBDQTAeFw0xNDAYMjAxMDAw\r\n\"
"MDBaFw0yNDAYMjAxMDAwMDBaMGYxZCZAJBgNVBAYTAkFMRkwFwYDVQQKEXBHbG9i\r\n\"
"YWxTaWduIG52LXNhMTwwOgYDVQQDEzNHbG9iYWxTaWduIE9yZ2FuaXphdGlubiBw\r\n\"
"YXpZGF0aW9uIENBIC0gU0hBMjU2IC0gRzlwggEiMA0GCSqGSIb3DQEBAAQAA4IB\r\n\"
"DwAwggEKAoIBAQDHDmW/I5N/zHClNDDIM/fsBOWphJykfVI+8DNIV0yKMCLkZc\r\n\"
"C33JiJ1Pi/D4nGyMVTXbv/Kz6vvjVudKRtkTIs021ZvBqOOWQ5PyDLzm+ebomchj\r\n\"
"SHh/VzZpGhkdwTtHufcKc1H/hgBKueuql6lfYygoKOhJJomlZeg0k9zfrtHOSewUj\r\n\"
"mxK1zusp36QUArkBpdSmnENkiN74fv7j9R7U/tjqORmMdlMJEkYuYlZCa7pnRxt\r\n\"
"Nw9KHjUgKOKv1CGLAcRfW4rY6uSa2EKTSDtc7p8zv4WtdufgPDWi2zZCHIKT3hl\r\n\"
"2pK8vjX5sT5J4BO/5ZS5glg4Qdz6V0rvbLxAgMBAAGjggEIMIBITAObgNVHQ8B\r\n\"
"Af8EBAMCAQYwEgYDVROTAQH/BAGwBgEB/wIBADAdBgNVHQ4EFgQUlt5h8b0cFilT\r\n\"
"HMDMfTuDAEDmGnwwRwYDVROgBEAwPjA8BgRVHSAAMDQwMgYIKwYBBQUHAgEWJmh0\r\n\"
"dHBzOi8vd3d3Lm91dG8uY29tL3JlcG9zaXRvcnkMDMGA1UdHwQsMCow\r\n\"
"KKAmoCSGImh0dHA6Ly9jcmwwZ2xvYmFsc2lnbi5uZXQvc9vdC5jcmwwPQYIKwYB\r\n\"
"BUHAQEEMTAvmC0GCCsGAQUFBzABhiFodHRwOi8vb2Nzc5nbG9iYWxzaWduLmNv\r\n\"
"bS9yb290cjEwHwYDVROjBBgwFoAUyHtmGkUNl8qJUC99BM00qP/8/UswDQYJKoZ\r\n\"
"l\r\n\"
```

```
"hvcNAQELBQADggEBAEyq7l69rgFgNzERhnF0tkZJyBAW/i9ilxerH4f4gu3K3w4s\r\n" \
"32R1juUYcqemOovJrKV3UPfnqTgol8UV6MqX+x+bRDmuo2wCld2Dkyy2VG7EQLy\r\n" \
"XN0cvfNVlg/UBsD84iOKJHDTu/B5GqdhclOKrwbFINihY9Bsrk8y1658GEV1BSl3\r\n" \
"30JAZGSGvip2CTFvHST0mdCF/vihCPnG9vHQWe3WVjwIKANnuvD58ZAWR65n5ryA\r\n" \
"SOICdJSXVWkkDoPWoc209fn5ikkodBpBocLTJlg1MGCUF7ThBCIxPTsvFwayuJ2G\r\n" \
"K1pp74P1S8SqtCr4fKGxhZSM9AyHDPsSQPhZSZg=\r\n" \
"-----END CERTIFICATE-----\r\n";
```

```
int ssl_id = 1; //SSL通道
int verify = 1; //单向认证
int timeout = 60; //握手超时时间
cm_ssl_setopt(ssl_id, CM_SSL_PARAM_VERIFY, &verify); //设置单向认证
cm_ssl_setopt(ssl_id, CM_SSL_PARAM_NEGOTIME, &timeout); //设置握手超时时间，单位秒。
cm_ssl_setopt(ssl_id, CM_SSL_PARAM_CA_CERT, http_ca); //单向认证设置根证书
```

若需要双向认证或无身份认证，将CM_SSL_PARAM_VERIFY设置成对应模式即可，然后根据需求添加客户端证书和密钥。若只是需要使用MQTTs或HTTPS等模块，只需完成SSL配置即可，无需进行之后的业务，具体如何实现MQTTs或HTTPS等模块的加密传输功能，请查看SDK中对应模块的示例代码或开发指导手册。

TCP连接

TCP连接使用ASOCKET接口。

ASOCKET接口进行TCP连接需要在eloop环境下执行，首先发送eloop事件。

```
/* 注册Event(将命令封装成Event发送到eloop执行) */
cm_eloop_event_handle_t cmd_OPEN_recv_event = cm_eloop_register_event(cm_asocket_eloop(),
__on_eloop_cmd_OPEN_recv_event, NULL);
/* 发送到eloop执行 */
cm_eloop_post_event(cmd_OPEN_recv_event);
```

底层代码启动时将自动创建ASOCKET所需的eloop线程，所以用户不必自行进行初始化，以上代码开机注网成功后即可直接使用，其中cm_asocket_eloop()即为获取默认的eloop实例，建议该实例勿用于用户其他业务，__on_eloop_cmd_OPEN_recv_event()为eloop事件回调函数，需要在该函数中进行TCP连接。

```
static void __on_eloop_cmd_OPEN_recv_event(cm_eloop_event_handle_t event, void *cb_param)
{
    /* 注销Event */
    cm_eloop_unregister_event(event);

    /* 创建socket */
    int sock = -1;
    sock = cm_asocket_open(AF_INET, SOCK_STREAM, IPPROTO_TCP, __on_asocket_event, NULL);

    if (-1 == sock)
    {
        cm_demo_printf("open failed\n");
        return;
    }

    cm_demo_printf("sock(%d) open\n", sock);

    test_ssl_sock = sock; //以一个变量保存创建的socket以便后续进行SSL连接
```

```

/* 配置连接参数并执行连接 */
struct sockaddr_in server_addr;
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = inet_addr(test_ssl_addr); //自行补充服务器地址
server_addr.sin_port = htons(test_ssl_port); //自行补充端口号
int ret = cm_asocket_connect(sock, (const struct sockaddr *)&server_addr, sizeof(server_addr));

if (ret < 0)
{
    if (EINPROGRESS == errno)
    {
        cm_demo_printf("sock(%d) open request success, wait connect...\n", sock);
    }

    else
    {
        cm_demo_printf("sock(%d) connect failed(%d)\n", sock, errno);
    }
}

else if (0 == ret)
{
    cm_demo_printf("sock(%d) connect_ok\n", sock);
}
}

```

以上代码中请注意对cm_asocket_connect()返回值的判断，若为0则连接成功，若小于0则需要判断errno，errno等于EINPROGRESS时为正在连接中，连接结果需要在回调函数中获知，回调函数即为cm_asocket_open()时传入的__on_asocket_event()，示例代码如下。

```

static void __on_asocket_event(int sock, cm_asocket_event_e event, void *user_param)
{
    switch (event)
    {
        /* TCP连接成功，之后才可进行SSL连接 */
        case CM_ASOCKET_EV_CONNECT_OK:
        {
            cm_demo_printf("sock(%d) connect_ok\n", sock);
            break;
        }

        case CM_ASOCKET_EV_CONNECT_FAIL:
        {
            cm_demo_printf("sock(%d) connect_fail\n", sock);
            break;
        }

        /* SSL握手阶段会上报读取事件，握手阶段不可进行数据读取 */
        case CM_ASOCKET_EV_RECV_IND:
        {
            /* 正在握手，不可执行读取 */
            if (1 == is_shaking_hands)
            {
                cm_demo_printf("ssl shaking hands\n");
                return;
            }
        }
    }
}

```


TODO: 触发其他线程执行读取任务, 该部分请用户自行完成, 读取任务尽量不在回调函数中进行, 回调函数中请勿阻塞或消耗过大资源。

```

}

/*
 *
 */
break;
}

case CM_ASOCKET_EV_SEND_IND:
{
    cm_demo_printf("sock(%d) send_ind\n", sock);
    break;
}

case CM_ASOCKET_EV_ACCEPT_IND:
{
    cm_demo_printf("sock(%d) accept_ind\n", sock);
    break;
}

case CM_ASOCKET_EV_ERROR_IND:
{
    /* 获取socket错误码 */
    int sock_error = 0;
    socklen_t opt_len = sizeof(sock_error);
    cm_asocket_getsockopt(sock, SOL_SOCKET, SO_ERROR, &sock_error, &opt_len);
    cm_demo_printf("sock(%d) error_ind: sock_error(%d)\n", sock, sock_error);

    if (ECONNABORTED == sock_error)
    {
        /* Connection aborted */
        cm_demo_printf("sock(%d) error_ind: Connection aborted\n", sock);
    }

    else if (ECONNRESET == sock_error)
    {
        /* Connection reset */
        cm_demo_printf("sock(%d) error_ind: Connection reset\n", sock);
    }

    else if (ENOTCONN == sock_error)
    {
        /* Connection closed */
        cm_demo_printf("sock(%d) error_ind: Connection closed\n", sock);
    }

    break;
}

default:
{
    break;
}

```



```

}
}

```

以上代码为ASOCKET接口的回调函数示例，进行SSL握手时不可进行数据读取，回调函数中尽量不进行阻塞任务或消耗过大资源。

SSL连接

ASOCKET接口成功完成TCP连接后，才可进行SSL连接，使用`cm_ssl_conn()`进行握手连接，该接口是阻塞接口，连接结果可直接从返回值判断，返回0则代表成功，否则为失败。握手超时时间由之前示例中的`cm_ssl_setopt()`进行设置。

```

/* 开始握手连接，握手阶段不可进行数据读取，使用标志位控制。cm_ssl_conn是同步接口，直接返回握手结果 */
is_shaking_hands = 1;
int ret = cm_ssl_conn(&test_ssl_ctx, ssl_id, test_ssl_sock, 0);
is_shaking_hands = 0;

cm_demo_printf("ssl connect ret %d\n", ret);

```

如上示例所示，示例中以`is_shaking_hands`作为握手标志位，在ASOCKET回调函数中控制业务逻辑。`test_ssl_ctx`是SSL创建后的句柄，需要供SSL其他接口使用，创建全局变量即可`static cm_ssl_ctx_t *test_ssl_ctx = NULL;`。`test_ssl_sock`是TCP连接时使用`cm_asocket_open()`返回的标识符。`ssl_id`是选择使用哪路通道的SSL。`cm_ssl_conn()`的最后一个参数是配置读取数据时的超时时间，既执行`cm_ssl_read()`时的超时时间。

SSL发送

使用`cm_ssl_write()`进行数据发送。

```

int send_len = 0;
int ret = 0;

char send_buffer[128] = "hello";
int len = strlen(send_buffer);

while(send_len < len)
{
    ret = cm_ssl_write(test_ssl_ctx, (void*)(send_buffer + send_len), len - send_len);

    if(ret == MBEDTLS_ERR_SSL_WANT_WRITE || ret == MBEDTLS_ERR_SSL_WANT_READ)
    //MBEDTLS正在等待发送或等待接收
    {
        continue;
    }

    if(ret >= 0) //发送成功
    {
        send_len += ret;
    }

    else //发送异常
    {
        send_len = ret;
    }
}

```

```
break;
}
}
```

如上示例代码所示，test_ssl_ctx即为执行cm_ssl_conn()连接成功后的句柄。

SSL接收

使用cm_ssl_read()进行数据读取。当ASOCKET的回调函数触发接收事件CM_ASOCKET_EV_RECV_IND时即可进行读取，但是握手阶段不能进行读取。

```
uint8_t recv_buf[128] = {0};
int ret = 0;

while(1)
{
    ret = cm_ssl_read(test_ssl_ctx, (void*)recv_buf, 128);

    if (ret > 0)
    {
        cm_demo_printf("ssl read data=%s\n", recv_buf);
    }

    else
    {
        cm_demo_printf("ssl read ret=%d\n", ret);
        break;
    }

    if (cm_ssl_check_pending(test_ssl_ctx) == 0) //查询ssl缓存中是否有未读数据
    {
        cm_demo_printf("ssl no pending data");
        break;
    }
}
```

如上示例代码所示，test_ssl_ctx即为执行cm_ssl_conn()连接成功后的句柄。

关闭连接

关闭连接时需要先关闭SSL连接，再关闭socket连接。关闭SSL连接直接执行cm_ssl_close()即可。

```
cm_ssl_close(&test_ssl_ctx);
```

关闭socket连接时，由于我们是使用ASOCKET接口，需要在eloop环境下关闭socket连接，首先发送eloop事件。

```
/* 注册Event(将命令封装成Event发送到eloop执行) */
cm_eloop_event_handle_t cmd_CLOSE_recv_event = cm_eloop_register_event(cm_asocket_eloop(),
__on_eloop_cmd_CLOSE_recv_event, NULL);
/* 发送到eloop执行 */
cm_eloop_post_event(cmd_CLOSE_recv_event);
```

__on_eloop_cmd_CLOSE_recv_event()为eloop事件回调函数，需要在该函数中进行关闭socket。

```
static void __on_eloop_cmd_CLOSE_recv_event(cm_eloop_event_handle_t event, void *cb_param)
{
    /* 注销Event */
    cm_eloop_unregister_event(event);

    /* 关闭socket */
    int ret = cm_asocket_close(test_ssl_sock);

    if (0 == ret)
    {
        cm_demo_printf("sock(%d) close success\n", test_ssl_sock);
    }

    else
    {
        cm_demo_printf("sock(%d) close error(%d)\n", test_ssl_sock, errno);
    }
}
```



中国移动
China Mobile

5. 编程设计注意

使用SDK提供的SSL接口进行开发时需注意以下事项：

- 证书数据大小不能超过8192字节。
- 握手阶段不可进行数据读取。
- 证书格式请参照本文档中示例。
- `cm_ssl_read()`返回的读取数据长度是解密后的长度，与`socket`接口获取待读取数据长度不同。
- 证书将保存于文件系统，设置前请先保证文件系统空间足够。
- 使用SSL接口进行MQTT或HTTPS等其他模块的加密传输时，只需要用`cm_ssl_setopt()`接口完成指定通道的数据配置即可，具体怎么实现请参考其他模块的示例程序，例如`cm_demo_mqtt.c`。

6. 附录

Table 2. 缩略语

缩写	英文全称	中文解释
SSL	Secure Sockets Layer	安全套接字协议